

Lab 03 — Scalable Loosely-Coupled Architecture

Route traffic by path, decouple producers from consumers with a message queue, and prove a failed backend does not take the frontend down.

Lab environment: Docker Desktop (free local simulation) — runs locally, no AWS account needed

Estimated time: 35–40 minutes

SAA-C03 domain: Design Resilient Architectures

Overview

Local component	AWS equivalent
lab3-lb (NGINX, port 8083)	Application Load Balancer with path-based routing
lab3-front	Web tier target group
lab3-api (/api/)	API tier target group (e.g. an ECS service)
lab3-rabbit (RabbitMQ)	Amazon SQS / Amazon MQ

The RabbitMQ management console is also published at <http://localhost:15683> (default login `guest` / `guest`).

Step 1 — Get the lab source and start the lab

The repo lives in `~/Downloads/aws-saa`. The paths are absolute, so these commands work from any folder. `git clone` is needed only once: if the repo is already there it prints `already exists`, and the next line still takes you into this lab.

```
cd ~/Downloads
git clone https://github.com/mohanpothula/aws-saa.git
cd ~/Downloads/aws-saa/labs/lab-03-loosely-coupled-architecture
docker compose up -d
docker compose ps
```

Step 2 — Path-based routing

Task: Test the frontend and API paths.

```
curl -w '\n' http://localhost:8083/
curl -w '\n' http://localhost:8083/api/
```

Expected result: Two different answers from the same address: `/` returns `<h1>FRONTEND</h1>` (the web tier) and `/api/` returns `API BACKEND` (the API tier).

Step 3 — Create a message queue

Task: Create an `orders` queue.

```
docker exec lab3-rabbit rabbitmqadmin declare queue name=orders durable=true
docker exec lab3-rabbit rabbitmqadmin list queues
```

Expected result: `orders` is listed.

Step 4 — Publish a message

Task: Send `ORDER-1001`.

```
docker exec lab3-rabbit rabbitmqadmin publish exchange=amq.default routing_key=orders
payload=ORDER-1001
```

Expected result: **Message published.**

Step 5 — Consume the message

Task: Read the queued order.

```
docker exec lab3-rabbit rabbitmqadmin get queue=orders ackmode=ack_requeue_false
```

Expected result: **ORDER-1001** is returned.

Step 6 — Failure isolation

Task: Stop the API and prove the frontend still works.

```
docker compose stop api
curl -w '\n' http://localhost:8083/
```

Expected result: **<h1>FRONTEND</h1>** is still returned: the frontend stays up while the API is down.

Verification

Check	Expected
Step 2 — Path-based routing	Two different answers from the same address: / returns <h1>FRONTEND</h1> (the web tier) and /api/ returns API BACKEND (the API tier)
Step 3 — Create a message queue	orders is listed
Step 4 — Publish a message	Message published
Step 5 — Consume the message	ORDER-1001 is returned
Step 6 — Failure isolation	<h1>FRONTEND</h1> is still returned: the frontend stays up while the API is down

Completion check

- [] Every validation result demonstrated to the trainer
- [] You can say which AWS service each local component represents
- [] You can explain the failure condition tested in Step 6

Cleanup

```
docker compose down -v
```

-v also removes the lab's volumes, so the next run starts from a clean state.

Reference

- Lab source code:
<https://github.com/mohanpothula/aws-saa/tree/main/labs/lab-03-loosely-coupled-architecture>
- Original student guide: **courseware/AWS_SAA_Lab3_Student_Guide.docx**
- [What is an Application Load Balancer?](#)
- [ALB listener rule conditions \(path-based routing\)](#)
- [Amazon SQS Developer Guide](#)
- [Amazon MQ \(managed RabbitMQ\)](#)
- [Amazon ECS Developer Guide](#)
- [NGINX proxy module \(proxy_pass, proxy_cache\)](#)

- [RabbitMQ management CLI \(rabbitmqadmin\)](#)
- [hashicorp/http-echo](#)